



Informe Final

API REST Empresarial

Integración de Spring Boot, Clean Architecture y
DevSecOps bajo el Modelo de Calidad de McCall

**UNIVERSIDAD DE LAS
FUERZAS ARMADAS ESPE**
Departamento de Ciencias de la Computación
Aseguramiento de la Calidad de Software

Cafeteria Don Gato

Sistema de Gestión de Inventario

Docente: Ing. Diego L. Gamboa
dlgamboa1@espe.edu.ec | **Fecha:** 13 de mayo de 2026

Equipo de trabajo

Mesias Mariscal Gabriel Murillo
Denise Rea Julio Viche

Spring Boot 4.0.6

17 endpoints

146 tests OK

JWT + McCall

Documento final basado en PDCA, Modelo de McCall,
Clean Architecture, pruebas automatizadas, cobertura Ja-
CoCo, Quality Scoring Engine y evidencias DevSecOps.

Índice

1. Introduccion	1
1.1. Contexto del Problema	1
1.2. Objetivos	1
1.3. Alcance del MVP	1
2. Metodologia	1
2.1. Aplicacion del Ciclo PDCA	2
2.2. Modelo de McCall	2
2.3. Trazabilidad entre Requisitos y Calidad	3
2.4. Clean Architecture	3
2.5. Matriz PDCA de Seguimiento	3
2.6. Concordancia entre Matriz y Evidencia Tecnica	4
3. Resultados	4
3.1. Metricas de Calidad Obtenidas	4
3.2. Quality Score Calculado	5
3.3. Evidencias de Ejecucion	6
4. Discusion	7
4.1. Factores McCall mas Criticos	7
4.2. Limitaciones Encontradas	8
4.3. Decisiones Arquitectonicas	8
4.4. Comparativa con Enfoques Alternativos	8
5. Conclusiones	9
5.1. Lecciones Aprendidas sobre Calidad Medible	9
5.2. Recomendaciones para Escalar el Framework	9
5.3. Trabajo Futuro	9

1 Introduccion

1.1 Contexto del Problema

La cafeteria Don Gato requiere controlar productos, categorias y movimientos de inventario de forma segura, trazable y consistente. En este tipo de proceso, una salida mal registrada, una categoria duplicada o un producto actualizado sin control puede afectar la disponibilidad del negocio, producir compras innecesarias y disminuir la confiabilidad de la informacion operativa.

El proyecto atiende esta necesidad mediante una API REST versionada para la gestion de inventario. La solucion separa las consultas publicas, necesarias para el frontend, de las operaciones administrativas protegidas por autentificacion JWT. Esta decision permite que el sistema mantenga una experiencia de usuario sencilla sin renunciar a controles de integridad para las acciones sensibles.

1.2 Objetivos

El objetivo general es construir una API REST empresarial educativa para la gestion de inventario de la cafeteria Don Gato, aplicando principios de Clean Architecture, seguridad JWT, pruebas automatizadas y evaluacion de calidad bajo el Modelo de McCall.

Los objetivos especificos son:

- implementar los modulos de autentificacion, categorias, productos y movimientos de stock;
- separar dominio, aplicacion, infraestructura e interfaces para mejorar mantenibilidad y testabilidad;
- proteger operaciones administrativas mediante tokens JWT;
- validar el backend con pruebas automatizadas y cobertura JaCoCo;
- consolidar resultados de calidad mediante un Quality Scoring Engine;
- relacionar las decisiones tecnicas con el ciclo PDCA y los factores de McCall.

1.3 Alcance del MVP

El MVP cubre las funciones necesarias para operar el inventario base de la cafeteria. No busca resolver todos los escenarios comerciales futuros, sino entregar una base tecnicamente solida que pueda crecer hacia roles, reportes, analitica y documentacion viva de API.

Tabla 1: Alcance funcional del MVP

Modulo	Alcance implementado
Autenticacion	Inicio de sesion con JWT y respuesta tipo Bearer.
Categorias	Creacion, listado, busqueda por id, actualizacion y eliminacion.
Productos	Creacion, listado, busqueda por id, filtrado por categoria, filtrado por estado, busqueda por nombre, actualizacion y eliminacion.
Movimientos de stock	Registro de entradas y salidas, listado general y consulta por producto.
Seguridad	Consultas publicas para productos y categorias; escritura y movimientos protegidos por JWT.
Calidad	Pruebas con JUnit, Mockito, H2 y Spring Security Test; cobertura JaCoCo; configuracion OWASP y JMeter.

La calidad se considero desde el diseno. Las reglas de negocio se aislaron en el dominio, los casos de uso se hicieron testeables, la API se versiono bajo /api/v1, la seguridad se configuro por rutas y las metricas se conectaron con el Quality Score.

2 Metodologia

2.1 Aplicacion del Ciclo PDCA

El desarrollo se organizo con el ciclo **Plan-Do-Check-Act**. En la fase de planificacion se definieron el problema, el alcance y los criterios de calidad. En la fase de ejecucion se implementaron las capas del sistema y los controles de seguridad. En la fase de verificacion se ejecutaron pruebas y metricas. En la fase de actuacion se identificaron mejoras necesarias para fortalecer el proceso DevSecOps.

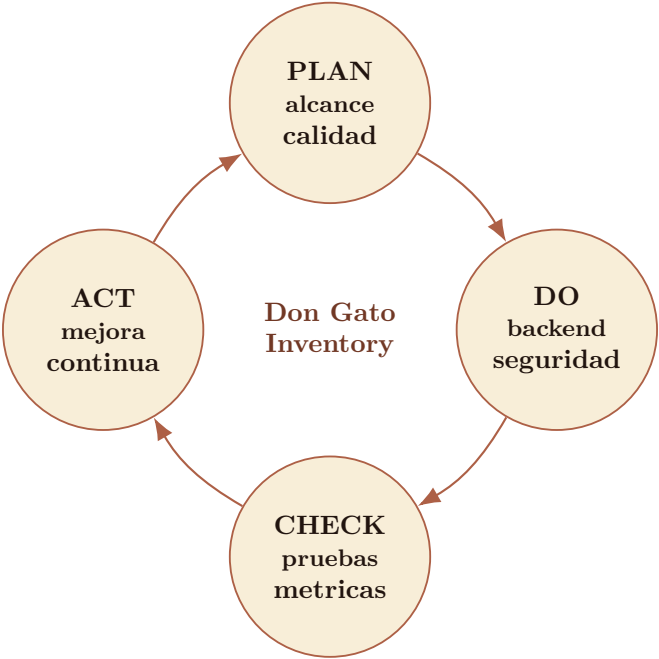


Figura 1: Ciclo PDCA aplicado al proyecto

2.2 Modelo de McCall

El Modelo de McCall se uso como marco de evaluacion porque permite relacionar requisitos tecnicos con atributos de calidad. En este proyecto se priorizaron Correctness, Reliability, Integrity, Maintainability, Testability, Portability e Interoperability.

Tabla 2: Factores de McCall aplicados al proyecto

Perspectiva	Factores	Aplicacion en Don Gato Inventory
Operacion del producto	Correctness, Reliability, Efficiency, Integrity, Usability	Reglas de stock, respuestas HTTP, JWT, validaciones y experiencia del frontend.
Revision del producto	Maintainability, Flexibility, Testability	Clean Architecture, puertos, DTOs, mappers y pruebas automatizadas.
Transicion del producto	Portability, Reusability, Interoperability	Maven Wrapper, Docker, PostgreSQL y API REST versionada.

2.3 Trazabilidad entre Requisitos y Calidad

Tabla 3: Mapeo de requisitos a factores de McCall

Requisito	Factor	Metrica	Validacion
Autenticacion JWT	Integrity	Rutas protegidas	Spring Security y pruebas de acceso.
Reglas de stock	Correctness	Cero defectos funcionales en pruebas	JUnit y Mockito.
Cobertura alta	Testability	Mayor o igual a 85 %	JaCoCo.
Arquitectura limpia	Maintainability	Separacion de capas	Revision de estructura.
Despliegue reproducible	Portability	Entorno ejecutable por configuracion	Maven Wrapper y Docker.
API versionada	Interoperability	17 endpoints REST	Controladores bajo /api/v1.
Rendimiento base	Efficiency	Objetivo menor a 200 ms	Plan JMeter.

2.4 Clean Architecture

La arquitectura separa el dominio de los detalles externos. Esta decision reduce acoplamiento y permite que las reglas del inventario puedan probarse sin depender de controladores HTTP, entidades JPA o configuracion de seguridad.

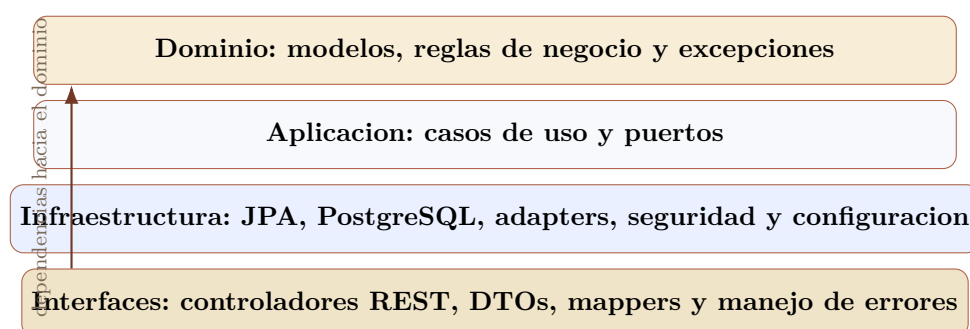


Figura 2: Separacion de capas usada en el backend

2.5 Matriz PDCA de Seguimiento

La hoja de calculo del equipo, identificada como *PROYECTO INVENTARIO*, se utiliza como evidencia de gestion del ciclo PDCA. Su informacion es compatible con el informe porque registra objetivo, responsables, avance, metricas y actividades por etapa. Para evitar duplicacion, el documento conserva solo los datos necesarios para demostrar trazabilidad.

Tabla 4: Datos principales de la matriz PDCA

Elemento	Registro principal
Proyecto	Inventario Don Gato - SQA.
Proceso	Arquitectura Hexagonal y SQA basado en McCall.
Ciclo	Ciclo 1.
Objetivo	Backend empresarial con arquitectura hexagonal, 17 endpoints validados, seguridad JWT y puntaje McCall superior a 80/100.
Metricas	Cobertura JaCoCo, latencia JMeter, vulnerabilidades OWASP y Quality Score.

Tabla 5: Avance por etapa PDCA registrado por el equipo

Etapa	Actividades registradas	Responsable	Avance
Planificar	Requerimientos, diseno arquitectonico, factores McCall y plan de pruebas.	Dev / QA	100 %
Hacer	Dominio, infraestructura, PostgreSQL, Docker, seguridad JWT y Quality Engine.	Dev	100 %
Verificar	Pruebas unitarias, validacion de carga y auditoria OWASP.	QA / Dev	100 %
Actuar	Quality gates, mejora de mantenibilidad, documentacion y cierre del ciclo.	Dev	100 %

2.6 Concordancia entre Matriz y Evidencia Tecnica

La matriz PDCA documenta el seguimiento del proceso, mientras que los reportes generados por herramientas aportan la evidencia tecnica. Cuando aparece una diferencia menor entre un registro manual y un reporte automatizado, el informe utiliza el valor automatizado y conserva la matriz como respaldo de planificacion y control.

Tabla 6: Conciliacion de datos entre matriz e informe

Indicador	Matriz PDCA	Evidencia usada en informe	Lectura
Cobertura	91.5 %	JaCoCo local: 90.77 % en lineas y 92 % en instrucciones; SonarCloud: 83.5 % en <i>overall code</i> .	Cumple en JaCoCo; SonarCloud aprueba el Quality Gate.
Latencia	185 ms	Plan JMeter orientado a validar respuestas menores a 200 ms.	Registro compatible.
Vulnerabilidades	0 criticas	SonarCloud reporta Security Rating A; OWASP Dependency Check queda configurado como gate.	Sin issues de seguridad abiertos en SonarCloud.
Pipeline	CI/CD operativo	GitHub Actions registra 35 corridas de workflow y ejecuciones recientes exitosas.	Captura disponible.
SonarCloud	23 issues iniciales	Resultado final: 0 open issues, 0.0 % duplicacion y Quality Gate aprobado.	Mejora verificada.
Avance PDCA	100 % por etapa	Informe cerrado con resultados y acciones de mejora.	Coherente.

La diferencia entre 91.5 %, 90.77 % y 83.5 % no representa una contradiccion: corresponden a fuentes y alcances distintos. La matriz PDCA registra el seguimiento del equipo, JaCoCo mide la cobertura local del backend y SonarCloud calcula cobertura sobre *overall code* con sus propias reglas de analisis. Para el calculo del Quality Score se utiliza 90.77 %, por ser el dato exacto disponible en el reporte JaCoCo local; para mantenibilidad se usa el estado final de SonarCloud con 0 open issues.

3 Resultados

3.1 Metricas de Calidad Obtenidas

La evidencia local fue generada el 13 de mayo de 2026 mediante la ejecucion de pruebas del backend. Los reportes de Surefire y JaCoCo indican que el proyecto cumple el umbral de cobertura establecido y no presenta fallos funcionales en la bateria automatizada disponible. Las capturas incorporadas despues de la mejora muestran tambien el estado de SonarCloud y GitHub Actions como evidencia visual del proceso DevSecOps.

Tabla 7: Resultados locales de pruebas y cobertura

Area	Indicador	Resultado
Pruebas automatizadas	Suites ejecutadas	24
Pruebas automatizadas	Casos ejecutados	146
Pruebas automatizadas	Fallos, errores y omitidas	0 / 0 / 0
Cobertura JaCoCo	Instrucciones	92 %
Cobertura JaCoCo	Ramas	78 %
Cobertura JaCoCo	Lineas	90.77 %
Cobertura JaCoCo	Metodos	91.52 %
Cobertura JaCoCo	Clases	90.63 %

Tabla 8: Indicadores de calidad y seguridad

Indicador	Resultado
Defectos en pruebas	No se registraron fallos ni errores en la ejecucion automatizada local.
SonarCloud Quality Gate	Estado <i>Passed</i> ; todas las condiciones del gate fueron aprobadas.
SonarCloud Open Issues	0 issues abiertos en el analisis final; inicialmente existian 23 hallazgos de mantenibilidad.
SonarCloud Coverage	83.5 % sobre <i>overall code</i> ; se reporta como lectura externa distinta de JaCoCo local.
SonarCloud Duplications	0.0 % de duplicacion.
Vulnerabilidades criticas	El umbral definido es 0 criticas; SonarCloud muestra Security Rating A y OWASP Dependency Check queda configurado para el ciclo <i>verify</i> .
Endpoints activos	17 endpoints REST versionados bajo <i>/api/v1</i> .
Seguridad	JWT para operaciones protegidas; consultas publicas controladas para productos y categorias.
Pipeline GitHub Actions	35 corridas de workflow registradas y ejecuciones recientes en estado exitoso.
Rendimiento	La matriz registra 185 ms y el plan JMeter mantiene el objetivo menor a 200 ms.

3.2 Quality Score Calculado

El Quality Scoring Engine pondera cuatro factores de McCall: Testability, Correctness, Maintainability e Integrity. El calculo usa cobertura de lineas como medida de testabilidad, resultados de pruebas como medida de correctitud, los issues abiertos de SonarCloud como referencia de mantenibilidad y el criterio de cero vulnerabilidades criticas como integridad.

$$Score = (Testability \times 0,30) + (Correctness \times 0,30) + (Maintainability \times 0,20) + (Integrity \times 0,20)$$

Tabla 9: Calculo del Quality Score

Factor	Valor	Peso	Aporte
Testability	90.77	0.30	27.23
Correctness	100.00	0.30	30.00
Maintainability	100.00	0.20	20.00
Integrity	100.00	0.20	20.00
Total			97.23 / 100

El resultado de 97.23/100 ubica al MVP en un nivel alto para el alcance academico del proyecto. La mejora respecto al calculo anterior se explica por la correccion de hallazgos de mantenibilidad en SonarCloud,

que paso de 23 issues abiertos a 0. Las mejoras pendientes se concentran en aumentar cobertura de ramas y publicar artefactos finales de OWASP y JMeter dentro del pipeline.

3.3 Evidencias de Ejecucion

Las evidencias se presentan de manera documental, asociando cada fuente con su resultado y con el factor McCall correspondiente. Este formato evita duplicar salidas de consola y mantiene el informe como documento formal.

Tabla 10: Evidencias tecnicas consolidadas

Fuente	Resultado registrado	Factor McCall
Surefire / Maven Test	146 pruebas ejecutadas, sin fallos, errores ni casos omitidos.	Testability, Reliability
JaCoCo	Cobertura de lineas de 90.77 % y cobertura de instrucciones de 92 %.	Testability
SonarCloud inicial	23 issues de mantenibilidad, con 19 de severidad media y 4 de severidad baja.	Maintainability
SonarCloud final	Quality Gate aprobado, 0 open issues, 0.0 % duplicacion, 83.5 % de cobertura overall code y Security Rating A.	Maintainability, Integrity
GitHub Actions	Workflow DevSecOps con 35 corridas registradas y ejecuciones recientes exitosas.	Portability, Reliability
Spring Security	Separacion entre rutas publicas y rutas protegidas con JWT.	Integrity
Matriz PDCA	Avance de 100 % en Planificar, Hacer, Verificar y Actuar.	Maintainability
JMeter	Registro de latencia de 185 ms en la matriz de seguimiento.	Efficiency
OWASP Dep. Check	Control configurado para validar cero vulnerabilidades criticas.	Integrity

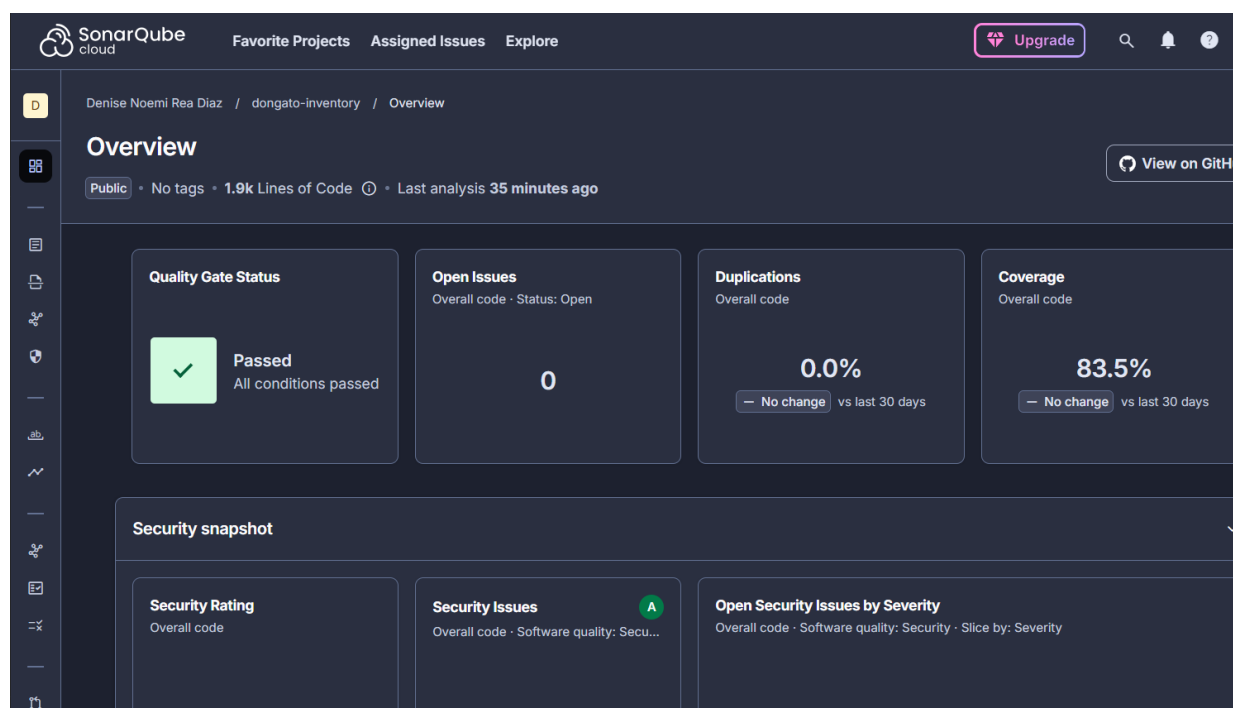


Figura 3: SonarCloud final: Quality Gate aprobado, 0 open issues, 0.0 % de duplicacion, 83.5 % de cobertura overall code y Security Rating A.

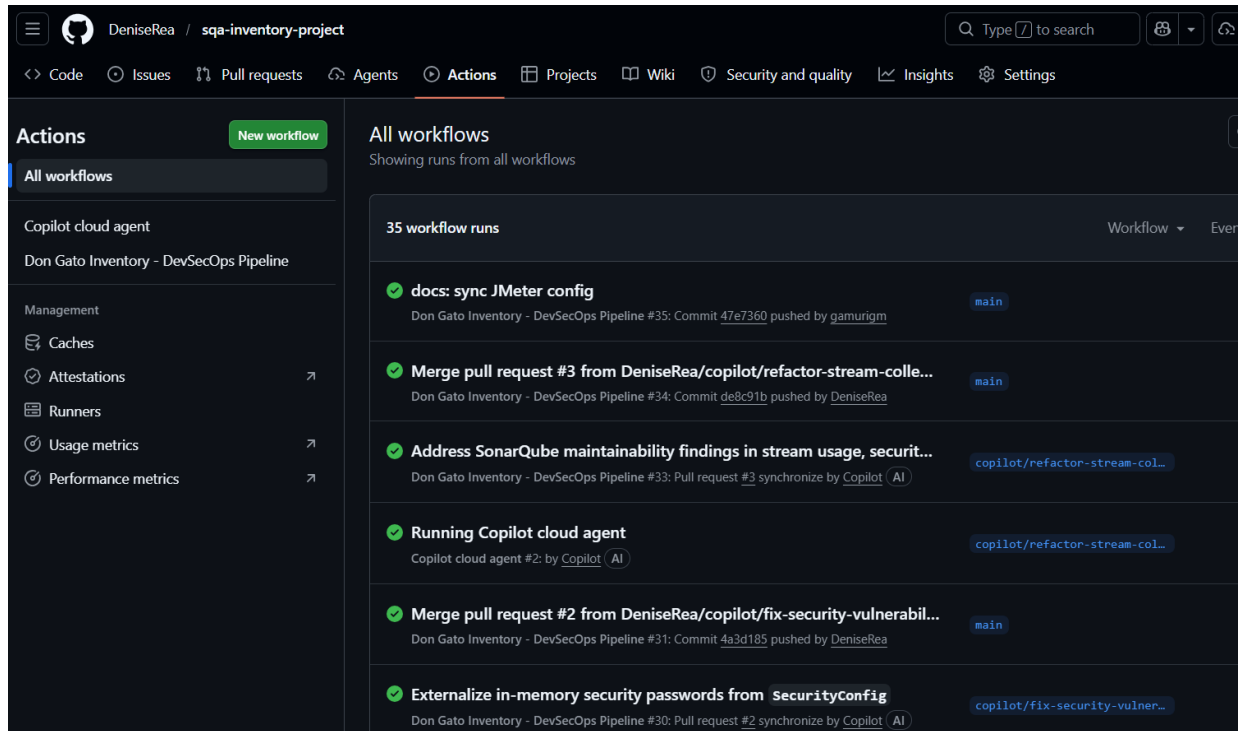


Figura 4: GitHub Actions: historial del workflow Don Gato Inventory - DevSecOps Pipeline con corridas recientes exitosas.

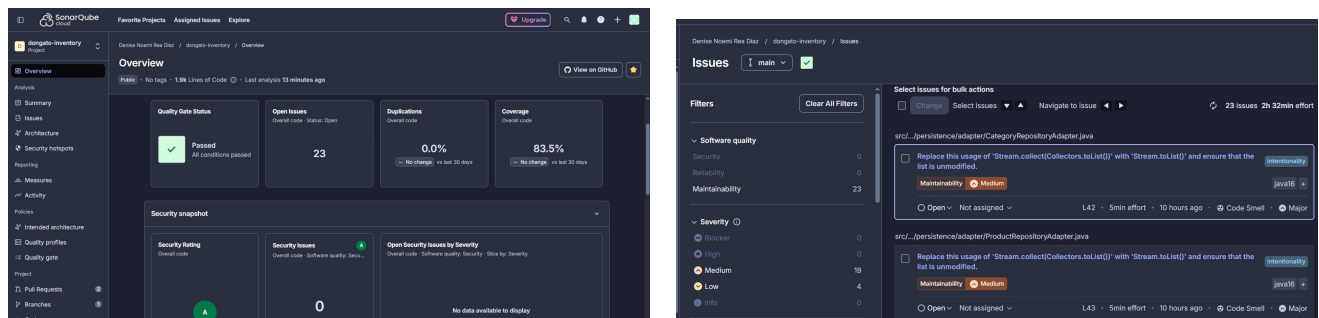


Figura 5: Estado inicial en SonarCloud: 23 issues de mantenibilidad, 19 de severidad media y 4 de severidad baja.

4 Discusion

4.1 Factores McCall mas Criticos

Los factores mas relevantes para este MVP son Integrity, Correctness, Testability y Maintainability. Integrity es prioritario porque el inventario no debe ser alterado por usuarios no autorizados. Correctness es critico porque una entrada o salida incorrecta afecta directamente el stock disponible. Testability permite evolucionar el sistema sin perder confiabilidad. Maintainability sostiene la escalabilidad del proyecto al separar responsabilidades por capas.

Tabla 11: Analisis de factores criticos

Factor	Justificacion	Evidencia en el proyecto
Integrity	Protege operaciones sensibles de inventario y administracion.	JWT, SecurityConfig y rutas protegidas.
Correctness	Evita inconsistencias de stock y respuestas funcionales incorrectas.	Reglas de dominio y pruebas automatizadas.
Testability	Reduce riesgo de regresiones durante cambios futuros.	146 pruebas y cobertura de lineas de 90.77 %.
Maintainability	Permite extender el sistema sin mezclar responsabilidades.	Clean Architecture, puertos y separacion por capas.
Portability	Facilita reproduccion del entorno en otros equipos.	Maven Wrapper, Dockerfile y Docker Compose.

4.2 Limitaciones Encontradas

Aunque el MVP cumple el objetivo academico, existen limitaciones que deben considerarse en una evolucion posterior. La cobertura de ramas es menor que la cobertura de lineas, por lo que conviene fortalecer pruebas sobre caminos alternos, errores de validacion y excepciones de seguridad. Ademas, OWASP y JMeter estan incorporados como parte del flujo de calidad, pero aun deben publicarse como artefactos finales del pipeline si se exige evidencia visual independiente.

Otra limitacion es que SonarCloud y JaCoCo no reportan exactamente el mismo porcentaje de cobertura porque miden alcances distintos. Por esta razon, el informe separa la cobertura local de JaCoCo, usada para el Quality Score, de la cobertura *overall code* presentada en SonarCloud. Finalmente, el MVP todavia no incluye paginacion avanzada, roles finos por perfil ni documentacion OpenAPI generada automaticamente.

4.3 Decisiones Arquitectonicas

La decision central fue usar Clean Architecture para proteger el dominio de detalles externos. Esta estructura permite que los casos de uso dependan de puertos y que la infraestructura implemente los adaptadores necesarios para persistencia, seguridad y comunicacion HTTP. Con ello, el proyecto mejora su mantenibilidad y conserva una base clara para agregar nuevos modulos.

La seguridad con JWT se eligio porque el frontend funciona como cliente desacoplado y necesita enviar credenciales en cada solicitud protegida mediante el encabezado **Authorization**. PostgreSQL se selecciono como base relacional principal por su uso empresarial, mientras que H2 permite ejecutar pruebas rapidas sin depender de infraestructura externa.

4.4 Comparativa con Enfoques Alternativos

Tabla 12: Comparativa de enfoques de desarrollo

Alternativa	Ventaja	Riesgo frente al enfoque usado
CRUD simple por capas	Menor tiempo inicial de implementacion.	Mayor acoplamiento entre controladores, entidades y repositorios.
MVP sin JWT	Configuracion inicial mas sencilla.	Operaciones criticas quedan expuestas o dependen de controles externos.
Desarrollo sin pruebas automatizadas	Avance aparente mas rapido.	Mayor probabilidad de regresiones y baja evidencia de calidad.
Base embebida como unica persistencia	Entorno local simple.	Menor cercania a un ambiente empresarial real.
Clean Architecture + DevSecOps	Mayor trazabilidad y escalabilidad.	Requiere mas disciplina tecnica y configuracion inicial.

El enfoque seleccionado exige mas estructura al inicio, pero ofrece mejor soporte para evolucionar el sistema, justificar la calidad con metricas y sostener futuras integraciones.

5 Conclusiones

5.1 Lecciones Aprendidas sobre Calidad Medible

El proyecto demuestra que la calidad debe definirse desde el inicio mediante criterios verificables. La cobertura, los resultados de pruebas, la seguridad por rutas y el Quality Score permiten transformar conceptos generales de calidad en evidencia concreta. Tambien se confirma que Clean Architecture facilita la trazabilidad entre requisitos, reglas de negocio y pruebas.

El Modelo de McCall resulto util para explicar por que se tomaron determinadas decisiones tecnicas. La seguridad JWT se relaciona con Integrity, las pruebas automatizadas con Testability y Reliability, la separacion de capas con Maintainability y la API versionada con Interoperability.

5.2 Recomendaciones para Escalar el Framework

Para fortalecer el proyecto se recomienda consolidar SonarCloud dentro de GitHub Actions como gate obligatorio, ejecutar `mvnw verify` antes de cada merge, publicar artifacts de JaCoCo, OWASP y JMeter en cada corrida de CI, e incorporar OpenAPI/Swagger como documentacion viva de endpoints.

Tambien se recomienda implementar roles y permisos mas finos para administrador, cajero y consulta; introducir Testcontainers para validar persistencia con PostgreSQL real; y extender el Quality Scoring Engine para leer reportes generados automaticamente, evitando calculos manuales.

5.3 Trabajo Futuro

Como trabajo futuro se plantea complementar McCall con ISO 25010, incorporar IA en testing para generar casos borde y datos sinteticos, medir p95/p99 de latencia en un ambiente controlado, agregar cache selectivo para consultas publicas de productos e integrar monitoreo continuo de calidad en el pipeline.

En conclusion, Don Gato Inventory entrega una API REST educativa con estructura empresarial: seguridad JWT, Clean Architecture, pruebas automatizadas, cobertura superior al umbral, trazabilidad PDCA, evaluacion por McCall y una ruta clara de mejora continua mediante DevSecOps.